

Dinamiche di Opinione su Reti Sociali

Documentazione del Modello Agent-Based

Indice

1	Panoramica	2
2	Requisiti e Installazione	2
2.1	Dipendenze	2
2.2	Avvio	2
2.3	Riproducibilità	2
3	Parametri Globali	2
4	Architettura del Modello	3
4.1	Classe <code>Messaggio</code>	3
4.2	Classe <code>UtenteRegolare</code>	3
4.3	Classe <code>StrategicInfluencer</code>	4
4.4	Classe <code>SocialNetworkModel</code>	4
5	Esperimenti	5
5.1	Parte 2 — Confronto tra Scenari	5
5.2	Parte 3 — Calibrazione di ε	5
5.3	Parte 4 — Resilienza	6
5.4	Parte 5 — Distribuzione Finale	6
6	Analisi Topologica	6
6.1	Evoluzione degli Archi	6
6.2	Distribuzione del Grado — CCDF	6
6.3	Sweep <code>m_edges</code> — Robustezza della Topologia	7
7	Esperimenti Scartati	7
7.1	Hub vs Posizionamento Casuale	7
8	Output Generati	8
9	Conclusioni	8

1 Panoramica

Il progetto simula la **polarizzazione delle opinioni** in una rete sociale artificiale mediante un modello agent-based (ABM) costruito con il framework **Mesa**. La rete è generata con l'algoritmo **Barabási–Albert** (BA), che produce una topologia *scale-free* con pochi nodi molto connessi (hub) e molti nodi periferici, analoga a piattaforme social reali.

Le opinioni degli agenti sono valori continui in $[0, 1]$:

- 0 = posizione estrema sinistra / Fazione B
- 1 = posizione estrema destra / Fazione A
- 0.5 = posizione moderata / neutrale

2 Requisiti e Installazione

2.1 Dipendenze

```
pip install mesa networkx matplotlib seaborn tqdm
```

Versioni testate: `mesa` ≥ 2.0 , `networkx` ≥ 3.0 , `matplotlib` ≥ 3.7 .

2.2 Avvio

Le celle del notebook vanno eseguite **nell'ordine in cui compaiono**. La **Parte 1** è prerequisito obbligatorio per tutte le parti successive: definisce le classi `UtenteRegolare`, `StrategicInfluencer` e `SocialNetworkModel` che vengono usate in ogni esperimento. Eseguire qualsiasi altra parte senza aver eseguito la Parte 1 causa un `NameError`.

2.3 Riproducibilità

Il seed è fissato a 42 per `numpy` e `random`:

```
SEED = 42
np.random.seed(SEED)
random.seed(SEED)
```

Impostare `SEED = None` per comportamento stocastico.

3 Parametri Globali

Tutti i parametri sono definiti come costanti all'inizio della Parte 1 del notebook e vengono letti dagli agenti e dal modello tramite attributi di `SocialNetworkModel`. Modificarli prima di eseguire gli esperimenti è sufficiente per variare il comportamento dell'intera simulazione senza toccare il codice delle classi.

Parametro	Valore	Descrizione
NUM_NODI	500	Nodi totali della rete
TOL	0.4	Tolleranza BC (ε in Deffuant 2000)
STEPS	150	Step temporali per simulazione
N_RUNS	12	Run indipendenti per stimare la varianza
M_EDGES	5	Archi aggiunti per nodo nella rete BA
SIGMA	0.15	Dispersione iniziale delle opinioni attorno a 0.5
ENGAGEMENT_WEIGHT	0.4	Peso della viralità nella filter bubble
P_REWIRE	0.05	Probabilità di rewiring per step
REWIRE_THRESHOLD	0.6	Soglia di distanza per tagliare un arco
BACKFIRE_THRESHOLD	0.8	Soglia per attivare il backfire effect
BACKFIRE_STRENGTH	0.05	Intensità del backfire
EPSILON	0.008	Spostamento per step dell'influencer Overton

4 Architettura del Modello

4.1 Classe Messaggio

Messaggio rappresenta un singolo contenuto che circola nella rete, analogo a un post o a un articolo condiviso su un social network. Ogni messaggio ha tre attributi:

- **id**: identificatore UUID univoco, usato per evitare che lo stesso messaggio venga processato più volte dallo stesso agente
- **valore**: opinione associata al contenuto, un numero in $[0, 1]$ che rappresenta la posizione ideologica del messaggio
- **colore**: etichetta della fazione di origine ('Rosso' o 'Blu'), usata solo per la visualizzazione

I messaggi vengono creati dagli **StrategicInfluencer** ad ogni step e si propagano di vicino in vicino attraverso la rete, venendo accettati o rifiutati dagli utenti in base alla loro tolleranza.

4.2 Classe UtenteRegolare

Agente BC con tre meccanismi algoritmici opzionali, tutti controllati da flag del modello.

Attributi principali

- **opinione**: valore corrente in $[0, 1]$
- **tolleranza**: soglia individuale per accettare opinioni diverse
- **rate_convergenza**: velocità di aggiornamento dell'opinione (default 0.5)
- **resonance_strength**: intensità della spinta verso gli estremi
- **messaggi_visti**: set degli ID già processati (evita duplicati)

Metodo _scegli_vicino — Filter Bubble Sceglie con quale vicino interagire tramite selezione pesata. Il peso di ogni vicino bilancia due forze:

$$w_v = \underbrace{\frac{1}{d + 0.01}}_{\text{affinità}} \cdot (1 - w) + \underbrace{d^2}_{\text{engagement}} \cdot w$$

dove $d = |\text{opinione}_{\text{self}} - \text{opinione}_v|$ e $w = \text{engagement_weight}$. Con $w = 0$ prevale la bolla di eco; con $w = 1$ prevale la viralità polarizzante.

Metodo _rewiring — Omofilia Dinamica Con probabilità p_{rewire} per step:

1. Taglia gli archi verso vicini con distanza di opinione $> \text{rewire_threshold}$
2. Aggiunge un arco verso un candidato con distanza $< \text{tolleranza}$

Metodo step — Aggiornamento per Tick Fase 1 — Propagazione messaggi: raccoglie i messaggi condivisi dai vicini, filtra quelli già visti e accetta solo quelli entro la soglia di tolleranza (BC applicato ai contenuti).

Fase 2 — Interazione diretta:

$$\Delta = \text{opinione}_{\text{altro}} - \text{opinione}_{\text{self}}$$

- Se $|\Delta| \leq \varepsilon$: convergenza BC + spinta verso l'estremo se l'altro è già estremo
- Se $|\Delta| > \text{backfire_threshold}$: effetto backfire, l'agente si allontana

4.3 Classe StrategicInfluencer

Agente speciale posizionato sugli hub della rete. Implementa la strategia **Overton Window**: parte da una posizione moderata (0.55 o 0.45) e si sposta di ε per step verso il polo target (1.0 o 0.0).

La modalità statica (`overtone=False`) posiziona l'influencer direttamente all'estremo fin dal primo tick — usata per confronto.

4.4 Classe SocialNetworkModel

Costruisce la rete BA e istanzia tutti gli agenti. Il parametro `modalita` controlla il posizionamento degli influencer:

- `'baseline'`: nessun influencer
- `'hub'`: influencer sui nodi a maggior grado
- `'casuale'`: influencer su nodi scelti uniformemente (non usato negli esperimenti finali)

Il `DataCollector` di Mesa raccoglie automaticamente ad ogni step le seguenti metriche, disponibili come `DataFrame` al termine di ogni run:

- `Opinione_Media`: media aritmetica delle opinioni di tutti gli `UtenteRegolare`
- `Std_Opinioni`: deviazione standard delle opinioni — misura diretta della polarizzazione
- `Pct_Catturati_Alto`: percentuale di agenti con opinione > 0.7 (radicalizzati Fazione A)

- **Pct_Estremo_Basso**: percentuale di agenti con opinione < 0.3 (radicalizzati Fazione B)
- **Pct_Centro**: percentuale di agenti moderati con opinione in $[0.3, 0.7]$
- **N_Archi**: numero totale di archi nel grafo — traccia l'evoluzione topologica

5 Esperimenti

5.1 Parte 2 — Confronto tra Scenari

Cinque scenari eseguiti con $N_RUNS=12$ run indipendenti, 150 step ciascuna:

Scenario	Influencer	Meccanismi
Baseline	Nessuno	BC puro
Meccanismi — No Influencer	Nessuno	Completi
Completo — Statico	1, fisso a 1.0 sull'hub	Completi
Completo — Overton	1, graduale $0.55 \rightarrow 1.0$	Completi
Due Fazioni — Overton	2 contrapposti sugli hub	Completi

Risultato principale Solo *Due Fazioni — Overton* produce polarizzazione reale ($Std \approx 0.4$). Lo scenario *Meccanismi — No Influencer* si sovrappone quasi perfettamente alla Baseline: i meccanismi algoritmici da soli non polarizzano in assenza di una forza esterna asimmetrica.

Statico vs Overton Lo scenario *Completo — Statico*, in cui l'influencer parte già a 1.0 fin dal primo tick, produce una deriva dell'opinione media verso l'alto ma **non condiziona efficacemente la rete**: la maggior parte degli agenti ha opinione iniziale attorno a 0.5, quindi la distanza dall'influencer supera immediatamente la soglia di tolleranza (0.4) e i messaggi vengono ignorati o innescano il backfire effect. Lo scenario *Completo — Overton*, partendo da 0.55 e avanzando di $\varepsilon = 0.008$ per step, rimane entro la finestra di accettazione degli utenti molto più a lungo, trascinando gradualmente le opinioni e producendo una radicalizzazione più profonda e stabile nel tempo.

5.2 Parte 3 — Calibrazione di ε

Sweep su 10 valori: $\varepsilon \in \{0.001, 0.005, 0.008, 0.01, 0.02, 0.05, 0.10, 0.20, 0.40, 0.60\}$.

Il picco di polarizzazione si osserva a $\varepsilon = 0.40$, coincidente con la soglia di tolleranza BC — l'influencer fa un salto pari all'intera finestra di accettazione in un tick, il che non è realistico come modello di deriva narrativa graduale.

Il valore $\varepsilon = 0.008$ è stato scelto perché:

- Cade nel plateau stabile dove l'estremizzazione è massima e la crescita è lineare
- Corrisponde a ~ 125 step per coprire $[0, 1]$, interpretabile come campagna graduale
- Mostra bassa varianza tra run

5.3 Parte 4 — Resilienza

L'influencer viene rimosso a step 40 o 75 e sostituito con un `UtenteRegolare` avente la stessa opinione corrente. Gli archi del nodo hub rimangono invariati.

Risultato principale La polarizzazione è **completamente auto-sostenuta**. Dopo la rimozione a step 75, la deviazione standard non diminuisce: Filter Bubble e Rewiring mantengono la divisione in modo autonomo. Il centro non si rigenera in nessuno scenario. Questo implica che la rimozione degli influencer non è sufficiente a de-polarizzare la rete.

5.4 Parte 5 — Distribuzione Finale

Istogramma in scala logaritmica delle opinioni a step 150. La distribuzione bimodale di *Due Fazioni* — *Overton* contrasta con la convergenza unimodale degli scenari a fazione singola, confermando che due forze contrapposte sono la condizione necessaria per la divisione stabile della rete.

6 Analisi Topologica

6.1 Evoluzione degli Archi

Con rewiring attivo il numero di archi cala rispetto alla baseline statica. Il meccanismo di taglio è sistematico (agisce su tutti i vicini distanti), mentre l'aggiunta è selettiva (un solo arco per step): il saldo netto è negativo, la rete diventa progressivamente più sparsa.

6.2 Distribuzione del Grado — CCDF

La **CCDF** (*Complementary Cumulative Distribution Function*, funzione di distribuzione cumulativa complementare) risponde alla domanda: *quanti nodi hanno almeno k connessioni?*

Formalmente, dato il grado K di un nodo scelto a caso:

$$P(K \geq k) = 1 - P(K < k)$$

Un valore $P(K \geq k) = 0.1$ a $k = 20$ significa che il 10% dei nodi ha 20 o più connessioni. Tracciata in scala log-log, una distribuzione a legge di potenza (come quella della rete BA originale) appare come una retta, rendendo immediato confrontare quanto la coda destra — cioè gli hub — cambia tra la rete iniziale e quella dopo il rewiring.

Con rewiring il grado massimo scende da ~ 100 a ~ 50 : gli hub perdono sistematicamente gli archi verso nodi con opinioni distanti. Al tempo stesso il grado medio sale perché i nodi intermedi guadagnano connessioni all'interno dei propri cluster omofili.

Il risultato è una rete **più frammentata**: non più pochi hub dominanti con connessioni globali, ma cluster densi e separati. Questo è il meccanismo strutturale che rende la polarizzazione auto-sostenuta anche dopo la rimozione dell'influencer.

6.3 Sweep `m_edges` — Robustezza della Topologia

Il parametro m della rete BA (archi aggiunti per ogni nuovo nodo) controlla la densità iniziale della rete e l'entità del vantaggio strutturale degli hub. Per verificare che i risultati non dipendessero da questa scelta, è stato condotto uno sweep su $m \in \{3, 5, 8\}$, corrispondenti rispettivamente a reti sparse (~ 1500 archi), medie (~ 2500 , valore usato nel progetto) e dense (~ 4000 archi).

Le metriche finali (Std Dev e % radicalizzati) variano meno del 5% tra i tre valori: il modello è robusto alla densità della rete. Questo conferma che i risultati non sono un artefatto della topologia scelta, ma riflettono la dinamica dei meccanismi algoritmici. Il codice dello sweep è disponibile nella Parte 3 del notebook (`sim_medge_robustezza.png`).

7 Esperimenti Scartati

7.1 Hub vs Posizionamento Casuale

Durante lo sviluppo è stata implementata e testata una terza modalità di posizionamento dell'influencer: `modalita='casuale'`, in cui i nodi influencer vengono scelti uniformemente a caso tra tutti i nodi della rete invece che tra gli hub.

Motivazione originale Verificare se il vantaggio dell'influencer dipende dalla posizione strutturale (hub con molte connessioni) o dalla sola strategia Overton. L'ipotesi era che un influencer su hub avesse accesso diretto a una frazione maggiore della rete fin dal primo tick, accelerando la propagazione.

Risultati osservati Il posizionamento sull'hub produce polarizzazione più rapida nelle prime decine di step, poiché l'influencer raggiunge subito i vicini più connessi. Con posizionamento casuale la convergenza è più lenta ma raggiunge livelli comparabili a regime (~ 120 – 150 step). La differenza non è sufficiente a giustificare uno scenario separato nel confronto principale.

Perché scartato Lo scenario *hub vs casuale* aggiungeva complessità visiva senza modificare le conclusioni qualitative. Il focus è stato spostato sulla strategia (Overton vs Statico) piuttosto che sul posizionamento, che in un contesto reale è meno controllabile di quanto il modello suggerisca.

Il codice rimane nel modello come terzo ramo del condizionale in `SocialNetworkModel`:

```
else:    # casuale
    inf_pos = self.random.sample(range(num_nodi), n_fazioni)
```

e può essere riattivato aggiungendo `'casuale'` a `SCENARI`.

8 Output Generati

File	Contenuto
<code>sim_fazioni_confronto.png</code>	Evoluzione temporale delle 6 metriche per i 5 scenari
<code>sim_fazioni_snapshot.png</code>	Snapshot della rete a step 0, 30, 75, 150
<code>sim_epsilon_sweep.png</code>	Polarizzazione e composizione finale vs ε
<code>sim_medge_robustezza.png</code>	Robustezza al variare di <code>m_edges</code>
<code>sim_resilienza.png</code>	Metriche dopo rimozione dell'influencer
<code>sim_resilienza_snapshot.png</code>	Snapshot + distribuzione finale per i 3 scenari resilienza
<code>evoluzione_topologia.png</code>	Archi nel tempo, CCDF e snapshot topologici
<code>distribuzione_finale.png</code>	Istogrammi log delle opinioni finali per scenario

9 Conclusioni

Il modello mostra che la polarizzazione è un fenomeno emergente e auto-sostenuto:

1. I meccanismi algoritmici da soli (Filter Bubble, Rewiring, Backfire) non polarizzano in assenza di una forza esterna asimmetrica.
2. La strategia Overton è più efficace dell'influencer statico perché rimane entro la finestra di tolleranza degli utenti più a lungo.
3. Una volta instaurata, la polarizzazione persiste anche dopo la rimozione dell'influencer: la de-polarizzazione richiederebbe un intervento attivo sulla struttura della rete o sui meccanismi algoritmici stessi.
4. La rete si frammenta topologicamente in cluster omofili, rendendo la divisione strutturalmente stabile.